

# Extended Abstract

**Motivation** Reinforcement learning (RL) fine-tuning has yielded substantial gains in large language models (LLMs) across reasoning benchmarks, yet its internal effects remain poorly understood. Compared to supervised fine-tuning (SFT), RL (particularly GRPO-style optimization) achieves better generalization and robustness. However, the mechanisms by which RL shapes internal representations, especially in contrast to SFT, are still largely opaque. This project aims to illuminate how RL modifies internal LLM structure, with implications for alignment, interpretability, and future fine-tuning strategies.

**Method** We conduct a detailed mechanistic comparison between RL and SFT across two open-source models: Qwen3-1.7B and LLaMA-3.2-1B-Instruct, using the GSM8K math reasoning benchmark. Both models are fine-tuned over four epochs with SFT (next-token prediction) and GRPO (with and without KL regularization). We analyze representation drift via layer-wise weight norm shifts, singular value decomposition (SVD) rank retention, token-level attention redistribution, and output entropy trends during sampling.

**Implementation** Our training pipeline spans both SFT and RL (GRPO) modes, using `max_tokens=512` and consistent prompts to enable direct comparison. Structural metrics include Frobenius norm distance to base, retained matrix rank from SVD, attention heatmap deltas, and per-token entropy tracking. Evaluation is conducted on GSM8K with accuracy as the primary metric, alongside average completion lengths.

**Results** RL models consistently outperform SFT across models e.g., on Qwen3-1.7B, RL achieves up to 83.55% accuracy vs. 62.70% for SFT. RL completions are longer and more consistent, often using the full token budget. L2 and SVD analyses show SFT causes larger representational drift and higher rank inflation, indicating memorization. In contrast, RL preserves low-rank priors while refining attention toward solution-critical tokens (e.g., `\boxed{}`). Token-level entropy plots reveal that RL models explore broadly before collapsing confidently onto answers, whereas SFT exhibits early overconfidence and reduced adaptability.

**Discussion** Our findings support the hypothesis that base models already encode latent reasoning capabilities, and RL serves to "nudge" relevant circuits rather than overhaul them. SFT drives heavier, more global changes tied to memorization, while RL encourages sparse, targeted adaptation, particularly when KL regularization is used. These differences manifest in sharper attention, better token selectivity, and more stable entropy dynamics during decoding.

**Conclusion** This study shows that RL fine-tuning induces more efficient, localized, and interpretable internal changes than SFT. The reduced drift and focused adaptation promote generalization and robustness without sacrificing alignment. These insights motivate the use of RL for controllable model refinement and provide a pathway toward deeper mechanistic understanding of LLM behavior.

---

# Understanding the effect of RL on the internal representation of LLMs

---

**Rahul Chand**

Department of Computer Science  
Stanford University  
rahulsc@stanford.edu

**Arpandeeep Khatua**

Department of Computer Science  
Stanford University  
akhatua@stanford.edu

## Abstract

Reinforcement learning (RL) fine-tuning has emerged as a powerful method for improving large language models (LLMs), especially on reasoning-intensive tasks. However, the internal effects of RL compared to supervised fine-tuning (SFT) remain underexplored. In this work, we provide a mechanistic analysis of how RL alters internal model representations by comparing GRPO-based RL and SFT on two open-source models: Qwen3-1.7B and LLaMA-3.2-1B, using the GSM8K math benchmark. We track changes in weight norms, singular value spectra, attention distributions, and token-level entropy throughout training. We find that SFT induces greater structural drift, including inflated norms and rank (suggestive of memorization) while RL preserves base representations, sharpens attention on key answer tokens, and promotes longer, more confident completions. These findings suggest that RL fine-tuning facilitates efficient, selective adaptation with improved generalization, offering a more interpretable path for aligning LLMs to downstream tasks. Code Plots

## 1 Introduction

Large language models (LLMs) have become the cornerstone of modern natural language processing, achieving state-of-the-art performance across diverse tasks. Beyond standard supervised pretraining, two main strategies have emerged to further specialize LLMs: supervised fine-tuning (SFT) and reinforcement learning (RL). While SFT relies on next-token prediction from human demonstrations, RL methods particularly RL from human feedback (RLHF) optimize behavior via scalar rewards, enabling models to better follow instructions, reduce harmful outputs, and reason more effectively.

Recent RLHF-trained models, such as OpenAI’s chain-of-thought reasoning model o1 OpenAI (2024b) and DeepSeek-R1 Shao et al. (2024), have demonstrated significant improvements over their SFT counterparts. OpenAI o3 employs RL to generate long latent reasoning chains before responding and achieves breakthroughs on STEM benchmarks OpenAI (2024b,a). DeepSeek-R1, in contrast, uses a critic-free RL algorithm (GRPO) from scratch without any supervised fine-tuning to induce reasoning capabilities comparable to o3, and later adopts a cold-start SFT stage to improve readability DeepSeek-AI et al. (2025); DeepSeek-AI (2025).

Despite these advances, the inner workings of RL fine-tuning remain opaque. Existing research primarily evaluates final outputs (accuracy, latency, etc.) and neglects how internal representations evolve. This gap is critical because RLHF algorithms typically include KL-divergence penalties intended to constrain drift from a base model, suggesting that internal structure is changing but in unclear ways.

In this work, **we ask what internal representations shift during RL fine-tuning, and how do they compare to SFT?** We approach this mechanistically by combining performance evaluation

with structural analyses. Using 2 open source LLM models and GSM8K math problems Cobbe et al. (2021), we compare SFT-finetuned vs. RL-finetuned (via GRPO) models. We examine weight norms, singular values, attention patterns, activation statistics, and completion entropy to characterize internal drift and generalization.

Our early results indicate that SFT leads to greater rank and norm increases in feed-forward layers consistent with memorization while RL induces more conservative changes, preserving base representations and promoting generalizable behavior. The RL models also produce longer, more accurate completions, with lower entropy but sharper attention on key tokens. These findings help explain **why RL fine-tuning works**: it encourages efficient, structured adaptation rather than wholesale architectural change.

Understanding how and what shifts inside LLMs during RL fine-tuning is vital to advancing model alignment, allowing researchers to anticipate undesirable drift, calibrate confidence, and develop training regimes that align behavior while preserving interpretability and structural integrity.

## 2 Background and Related Work

### 2.1 Supervised Fine-Tuning in LLMs

Large pre-trained language models are typically adapted to downstream tasks or alignments via *supervised fine-tuning* (SFT). In this phase, the model continues training on human-labeled examples (e.g., question-answer pairs or demonstrations) to specialize its behavior (Ouyang et al., 2022; Wei et al., 2021). Early studies showed that fine-tuning on diverse, instruction-prefixed tasks broadens cross-task generalization. Instruction tuning approaches such as FLAN and T0 significantly improved zero-shot performance on unseen tasks (Wei et al., 2021). InstructGPT adopted a similar method, initially fine-tuning GPT-3 on curated instruction data before applying reinforcement learning from human feedback (RLHF) Ouyang et al. (2022).

SFT generally uses a cross-entropy loss to maximize likelihood of reference outputs:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y) \sim \mathcal{D}} \sum_{t=1}^T \log \pi_{\theta}(y_t | y_{<t}, x)$$

While effective, this loss can lead to overfitting, brittle generalization, and less diverse outputs. Li et al. (2024) propose GEM, an entropy-regularized objective encouraging flatter token distributions, which reduces overconfidence and improves output robustness.

### 2.2 Group Relative Policy Optimization (GRPO)

To address limitations in PPO-based fine-tuning such as the overhead of a large critic and instability with sparse rewards Shao et al. (2024) introduced **Group Relative Policy Optimization (GRPO)**. Instead of a learned value function, GRPO estimates advantages by comparing sampled outputs within a group:

$$A_i = \frac{R_{\phi}(\gamma_i) - \text{mean}(\mathcal{G})}{\text{std}(\mathcal{G})}, \quad \gamma_i \in \mathcal{G}$$

This group-based comparison avoids training a separate critic and uses a clipped surrogate loss with KL regularization:

$$\mathcal{L}_{\text{GRPO}}(\theta) = \mathcal{L}_{\text{Clip}}(\theta) - w_1 \text{KL}(\theta)$$

GRPO cuts memory usage roughly in half and simplifies training dynamics, aligning well with reward-based ranking. It enabled a 7B model to reach 51.7% accuracy on MATH without external tools Shao et al. (2024), and powered DeepSeek-R1-Zero to learn reasoning strategies via RL alone without supervised warm-up et al. (2024).

### 2.3 Interpretability Techniques for LLMs

A wide range of interpretability techniques have been developed to understand how LLMs represent and process information. **Probing** trains lightweight classifiers on hidden representations to extract features like part-of-speech and syntax Hewitt and Manning (2019); Belinkov and Glass (2019).

Although probing shows what is encoded, it does not explain how features affect outputs. For causal analysis, **activation patching** replaces parts of a model’s internal state with those from another input to identify which components are responsible for behavior Meng et al. (2022); Wang and et al. (2023).

More recent methods seek semantically meaningful directions in hidden states. **Sparse Autoencoders (SAEs)** learn sparse, linear decompositions that reconstruct activations while disentangling overlapping features Cunningham and et al. (2023). These latent factors tend to align more cleanly with human-understandable concepts than individual neurons. Other work focuses on attention mechanisms. **Attention rollout** and **attention flow** aggregate attention weights across layers to estimate token influence globally Abnar and Zuidema (2020). In parallel, **mechanistic interpretability** aims to reverse-engineer circuits within transformer architectures Olah et al. (2020), using a combination of feature tracing, weight path analysis, and intervention techniques.

## 2.4 Interpretability in RL-Finetuned LLMs

A key area of emerging interest is how RLHF impacts internal model representations. RLHF-tuned models often exhibit sharper attention distributions, with cross-attention focusing more narrowly on key prompt tokens Wu and et al. (2023). This sharpening corresponds to reduced entropy in next-token predictions, suggesting more confident or deterministic output behavior Li et al. (2024). KL-regularization in the RL objective acts to anchor the new policy to its pre-trained origin Ouyang et al. (2022); Shao et al. (2024), but aggressive tuning may still lead to *mode collapse*, where models always produce similar responses.

Empirical analysis reveals that later layers shift more significantly during RLHF than earlier ones, consistent with the architecture’s top-down adaptation. Metrics like **attention entropy** and **activation entropy** quantify these trends. Lower entropy often signals more focused processing, but may also reflect reduced diversity and model overconfidence Wu and et al. (2023). Interpretability tools applied to RLHF models aim to uncover how reward shaping reconfigures internal structure e.g., through specialized neurons or altered attention pathways. Understanding these internal changes is critical for ensuring that alignment improves helpfulness without sacrificing transparency, diversity, or factuality.

## 3 Method and Experiments

Table 1: Eval scores and average completion lengths across epochs for multiple models. The base (non-finetuned) model score for Qwen is **20.77** and for LLaMA is **48.67**.

| Method                    | Eval Score (%) |       |       |       | Avg Completion Length |     |     |     |
|---------------------------|----------------|-------|-------|-------|-----------------------|-----|-----|-----|
|                           | 1              | 2     | 3     | 4     | 1                     | 2   | 3   | 4   |
| Epoch                     |                |       |       |       |                       |     |     |     |
| Qwen3-1.7B RL (KL 0.04)   | 80.74          | 81.88 | 83.55 | 81.43 | 695                   | 695 | 696 | 695 |
| Qwen3-1.7B RL (no KL)     | 82.64          | 82.87 | 83.47 | 81.40 | 697                   | 697 | 697 | 697 |
| Qwen3-1.7B SFT            | 54.40          | 63.00 | 62.90 | 62.70 | 688                   | 583 | 649 | 629 |
| LLaMA-3.2 1B RL (KL 0.04) | 54.36          | 58.38 | 58.30 | 58.07 | 692                   | 692 | 692 | 671 |
| LLaMA-3.2 1B RL (no KL)   | 28.35          | 50.11 | 38.89 | 51.10 | 692                   | 692 | 692 | 692 |
| LLaMA-3.2 1B SFT          | 26.99          | 32.15 | 34.80 | 35.63 | 669                   | 580 | 531 | 501 |

To study the how RL and SFT change the internal representation of LLMs, we train two base models Qwen/Qwen3-1.7B and unsloth/Llama-3.2-1B-Instruct (which is an instruction finetuned version of Llama-3.2-1B). The reason we use the the instruction finetuned version for Llama is because the base model is poor at instruction following, therefore its hard to use GRPO with it (since its hard to verify answers since the model doesn’t follow the formating rules specified in prompt).

For dataset, we select the GSM8K Cobbe et al. (2021). GSM8K is a dataset of 8.5K high quality diverse grade school math word problems, with a train-test split of 7.5k and 1k. The reason we use a math dataset is because (i) We wanted to work in a verifiable domain (e.g. coding, maths) because it allows us to use RL methods such as GRPO, and we don’t have to use any non-deterministic reward models (ii) Its easier to interpret results in the context of MATH, e.g. seeing the model take two different trajectories when answering can be viewed as the model trying to solve a math problem via



two different methods, making such interpretable conclusions is tough when dealing with a more subjective datasets such as ones related to human preference.

For our experiment we train both base models for the same number of epochs (4). Additionally, for the GRPO training, we train the models with and without the KL-divergence term, this is to check how much our results are affected by KL-divergence. The performance of the base model and the SFT and RL trained models at each epoch is shown in Table 1. The evaluation metric here is accuracy (no. of correct answers divided by total questions). Also shown is the average completion length (the prompt length + the new tokens generated).

From our results there are few things we notice. First, RL performs much better than SFT, with  $\sim 20\%$  gap on the test set for Qwen and even larger  $\sim 24\%$  gap for Llama. RL on average also produces longer completions than SFT, in most cases using all the available tokens (i.e. given `max_tokens=512` budget, the completion generally uses all the 512 tokens). This is likely due to two factors (1) Once RL finds the correct answer it keeps repeating the same answer using the format provided in the prompt (e.g. `\boxed{\}`) and (2) In the GRPO training phase the model is incentivized to explore which often leads to longer and better answers, this is a phenomenon which has been observed previously as well, e.g. in the DeepSeek-R1 paper DeepSeek-AI et al. (2025). For Qwen, training with and without the KL-divergence term results in no distinguishable difference in the final performance. While for Llama, training with KL-divergence leads to better results ( $\sim 7\%$  gap). Note that for Llama-3.2 1B, SFT makes the model worse. This is likely because the base model itself has been finetuned for a long duration.

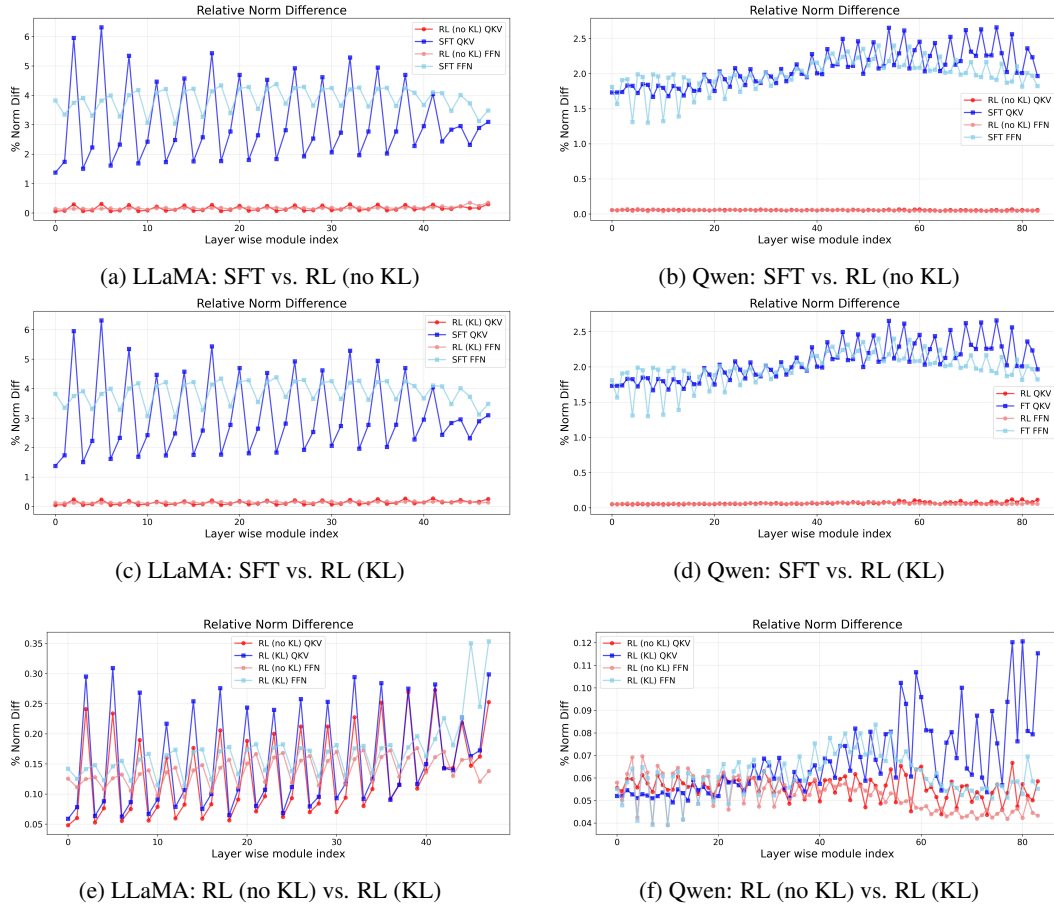


Figure 1: L2 distance from base model across training regimes for LLaMA (left) and Qwen (right). SFT causes larger shifts, while RL models (with or without KL) remain closer to the original weights.

### 3.1 Quantitative and Qualitative Results

To understand how SFT and RL differ in terms of how they affect internal representations we analyse the model and results from two perspectives (i) High-level view using the weights of model (ii) Finer-level view using individual tokens.

#### 3.1.1 High level analysis

**L2 distance:** To first understand how the different algorithms affect the model, we look at how far the weights of the finetuned models are from the base model. The results for this are shown in 1. SFT training leads to a larger deviation from the base model, whereas for RL (both with and without KL-divergence) the finetuned model remains really close to the base model.

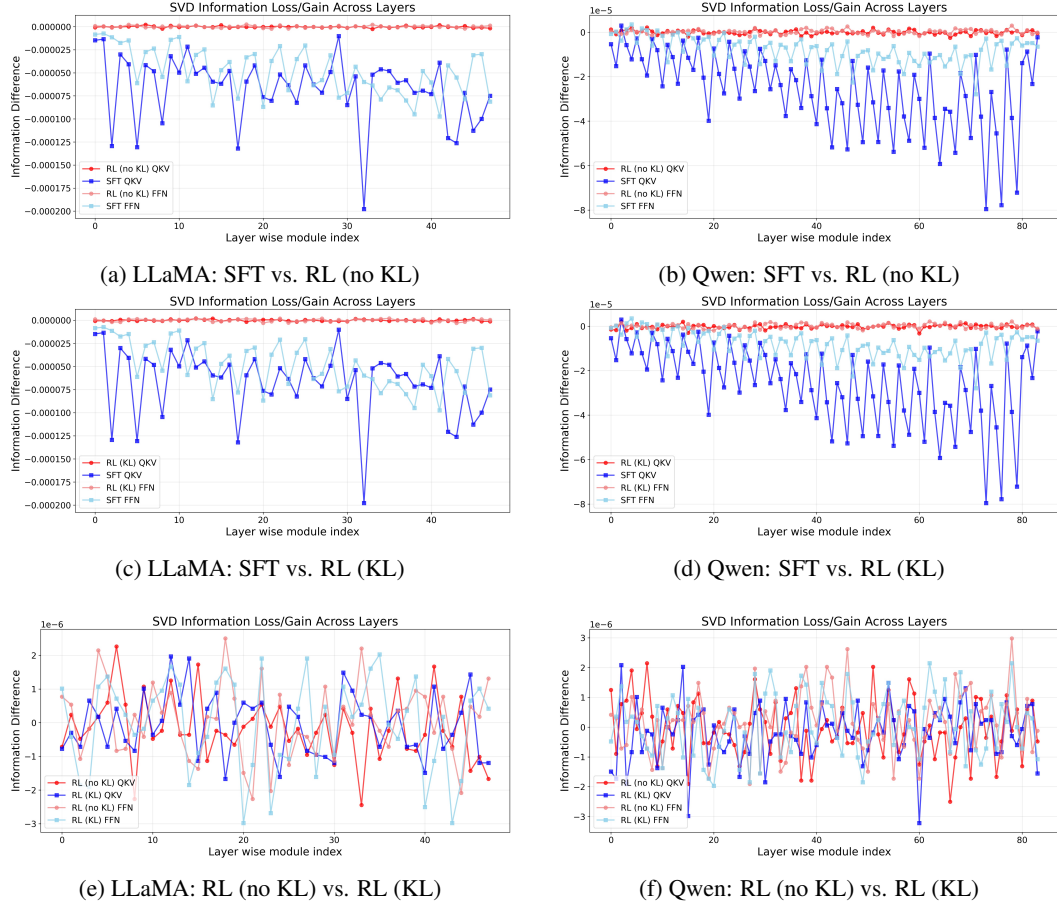


Figure 2: Difference in retained matrix information (via truncated SVD) across training regimes for LLaMA (left) and Qwen (right). SFT increases rank, while RL models tend to preserve or reduce it.

**Rank change:** We also calculate the rank of each matrix. A higher rank means more “information”, whereas a lower rank means the matrix stores less “information”, since it can be decomposed into a smaller one with only a minor loss in information. To measure this we first find the rank ( $K$ ) of the base model weight which accounts for 99% of information (using SVD). Let’s say for a  $Q$  (query) matrix ( $2048 \times 2048$ ),  $K = 2000$ , we then find out how much information does the same  $K$  top eigenvalues preserve in the finetuned (RL/SFT) weights and then subtract the two values. A negative value here means the rank increased during training. Why is this true? Because a negative value here means the same vectors now hold less information than they did previously. The reason we don’t just subtract the rank’s of the base and finetuned weights directly is because the rank might remain similar or change minimally (by 1) but the information can still spread or concentrate. We observe that SFT training leads to weights with higher rank than the base models as shown in Figure 2.

**How do we intuitively understand the results?** Our hypothesis for the figures and the results is that SFT learns "hardcoded" concepts, therefore, it performs worse on the test set but shows an increase in rank and norm (since it needs to store more information). In contrast, RL generalizes, therefore, it, on average, loses information (since general concepts require less information to store).

For example, consider the problem of adding numbers. We can have two models: one which learns many rules like  $2 + 2 = 4$ ,  $4 + 4 = 8$ ,  $8 + 10 = 18$ , etc., to solve the task. And another which learns just the general way to add and applies it. The first model stores more information (rules) but would perform worse on the test set, whereas the second model requires less information storage (just a few general rules) but achieves better test set accuracy.

Another way to understand this is that base models already have the "reasoning ability" to solve these problems and all RL does is slightly nudge/activate the specific circuits/weights in the right direction.

### 3.1.2 Token level analysis

**Self attention matrix:** In Figure 3 we plot the difference between the self-attention matrix of the finetuned model (RL/SFT) and the base model for a particular prompt. Similar to above, both the self attention pattern and the scores remain much closer in "RL vs. Pre" than in "SFT vs. Pre". Please note that, in Figure 3 the scales for first row ("SFT vs. Pre") has much higher range (0.1 to -0.1) than the scales for the other rows. Therefore, visually it might look like "RL vs. Pre" has more or simialr difference but it is not the case. Interestingly here the difference between KL and no KL is more prominent (especially in Llama), where self attention scores of model trained with KL, as expected, remain more closer to the base model.

**Per token attention pattern:** In Figure 4, we visualize the difference in attention received by each token in the prompt between the base and finetuned models. For a given self-attention matrix  $A \in \mathbb{R}^{N \times N}$ , where  $A_{ij}$  represents the attention paid by token  $i$  to token  $j$ , we compute the total attention received by each token  $j$  as the column-wise sum  $s_j = \sum_{i=1}^N A_{ij}$ . To mitigate the influence of the first token (which often acts an attention sink Barbero et al. (2025)), we discard  $s_1$  and normalize the remaining vector.

*How to read the figure?* Each row of the figure (marked as (a), (b) ...) shows 3 comparisons, between "SFT vs. Pre" where we subtract the vectors  $s$  of SFT trained model and base model, similarly, "RL vs Pre" and "RL vs SFT". Blue represents token being paid more attention to and red represents token being paid less attention to.

*How to interpret the figure?* One observation which remains consistent both across models and examples is that RL pays much more attention to the formatting token (`\boxed{\}`) than SFT. In fact, SFT pays less attention to it than even the base trained model (after normalization that is). This is likely due to two reasons (i) For GRPO the reward strictly depends on the right answer being within `\boxed{\}`, therefore the model tries to always output `\boxed{\}` and also outputs it multiple times (often repeating the answer) (ii) Given what we know from other results (e.g. small L2 difference) and our hypothesis that RL trained model only slightly nudge the model in the right direction, the per token attention pattern means that the RL tokens pays more attention to just a few tokens and these tokens are sufficient in improving performance.

Another observation which helps support the 2nd point above is that if you look at the attention difference in tokens which make up actual contents of the math problem (e.g. "Grace weights 25 pounds, Alex weights 2 pounds less than 4 times .."). You see that difference between RL and base model ("Pre") is small (smaller than between SFT and base). Ideally one would expect that since the RL model performs much better than the base, it either pays more attention or a different pattern of attention to these "logical tokens", but this is not the case. This is another observation supporting the theory that base models often contain the circuits/knowledge to solve these problems and RL only slightly nudges them in the correct direction.

**Entropy during sampling:** In Figure 5 we plot how the entropy ("sharpness") of tokens as they are sampled. More specifically,  $e_t$  is calculated as below. Note  $s_{t,j}$  is the probability of logit score (before softmax) of the  $j^{th}$  vocabulary word for the  $t^{th}$  token being sampled.

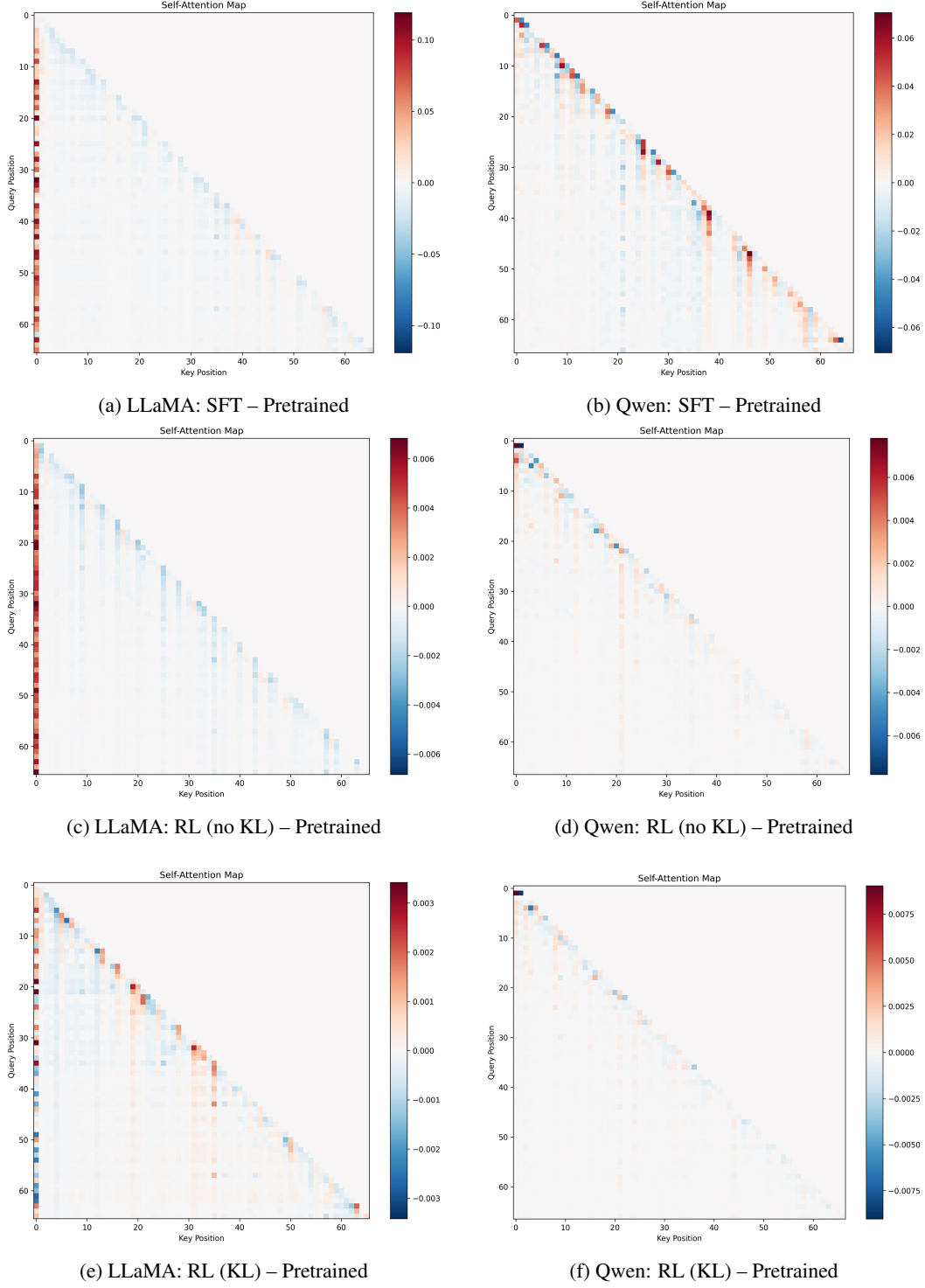


Figure 3: Difference in self-attention matrices between pretrained and finetuned checkpoints for a representative prompt. LLaMA (left) and Qwen (right) models are shown across SFT, RL (no KL), and RL (KL). RL-trained models retain attention structure more closely, especially with KL regularization.

Please solve this math problem step by step. Show your reasoning and always put the final number answer in \boxed{} format. Only put a single number answer in the box.

Problem: Grace weighs 125 pounds. Alex weighs 2 pounds less than 4 times what Grace weighs. What are their combined weights in pounds

Please solve this math problem step by step. Show your reasoning and always put the final number answer in \boxed{} format. Only put a single number answer in the box.

Problem: Grace weighs 125 pounds. Alex weighs 2 pounds less than 4 times what Grace weighs. What are their combined weights in pounds

Please solve this math problem step by step. Show your reasoning and always put the final number answer in \boxed{} format. Only put a single number answer in the box.

Problem: Grace weighs 125 pounds. Alex weighs 2 pounds less than 4 times what Grace weighs. What are their combined weights in pounds

(a) LLaMA: Change in attention received - SFT vs. Pretrained, RL vs. Pretrained, RL vs. SFT

Please solve this math problem step by step. Show your reasoning and always put the final number answer in \boxed{} format. Only put a single number answer in the box.

Problem: Grace weighs 125 pounds. Alex weighs 2 pounds less than 4 times what Grace weighs. What are their combined weights in pounds?

Please solve this math problem step by step. Show your reasoning and always put the final number answer in \boxed{} format. Only put a single number answer in the box.

Problem: Grace weighs 125 pounds. Alex weighs 2 pounds less than 4 times what Grace weighs. What are their combined weights in pounds?

Please solve this math problem step by step. Show your reasoning and always put the final number answer in \boxed{} format. Only put a single number answer in the box.

Problem: Grace weighs 125 pounds. Alex weighs 2 pounds less than 4 times what Grace weighs. What are their combined weights in pounds?

(b) Qwen: Change in attention received - SFT vs. Pretrained, RL vs. Pretrained, RL vs. SFT

Please solve this math problem step by step. Show your reasoning and always put the final number answer in \boxed{} format. Only put a single number answer in the box.

Problem: Tom gets 4 car washes a month. If each car wash costs \$15 how much does he pay in a year?

Please solve this math problem step by step. Show your reasoning and always put the final number answer in \boxed{} format. Only put a single number answer in the box.

Problem: Grace weighs 125 pounds. Alex weighs 2 pounds less than 4 times what Grace weighs. What are their combined weights in

Please solve this math problem step by step. Show your reasoning and always put the final number answer in \boxed{} format. Only put a single number answer in the box.

Problem: Tom gets 4 car washes a month. If each car wash costs \$15 how much does he pay in a year?

(c) LLaMA: Change in attention received - SFT vs. Pretrained, RL vs. Pretrained, RL vs. SFT

Please solve this math problem step by step. Show your reasoning and always put the final number answer in \boxed{} format. Only put a single number answer in the box.

Problem: Tom gets 4 car washes a month. If each car wash costs \$15 how much does he pay in a year?

Please solve this math problem step by step. Show your reasoning and always put the final number answer in \boxed{} format. Only put a single number answer in the box.

Problem: Tom gets 4 car washes a month. If each car wash costs \$15 how much does he pay in a year?

Please solve this math problem step by step. Show your reasoning and always put the final number answer in \boxed{} format. Only put a single number answer in the box.

Problem: Tom gets 4 car washes a month. If each car wash costs \$15 how much does he pay in a year?

(d) Qwen: Change in attention received - SFT vs. Pretrained, RL vs. Pretrained, RL vs. SFT

Figure 4: Per-token attention shift across training regimes for LLaMA and Qwen. Each subfigure shows the normalized difference in attention received by each token (excluding the first) between model variants. Blue indicates increased attention in the first model of the pair, red indicates reduced attention. Notably, RL consistently increases focus on task-critical formatting tokens like \boxed{}.

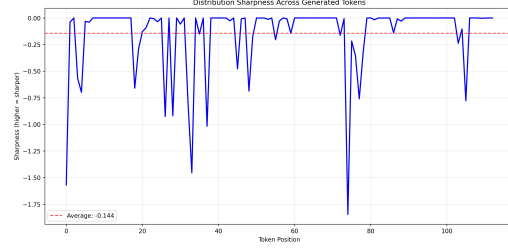
$$e_t = \sum_{v=1}^V p_{t,v} \log p_{t,v}, \quad \text{where } p_{t,v} = \frac{\exp(s_{t,v})}{\sum_{j=1}^V \exp(s_{t,j})} \quad (1)$$

The above equation is just the entropy (without the negative sign). A flatter, more uniform distribution means a higher negative value of  $e_t$ . A sharper peaked distribution (where only one word/token is highly probable) means  $e_t$  will be closer to 0.

There are two things we observe from Figure 5 (i) RL trained models are more exploratory in the initial phase of sampling their answers (they have higher negative values) while SFT models start out being much more confident. This is due to fundamental difference in how RL and SFT works, RL

Grace's weight is G = <<125=125>>125 pounds  
 Alex's weight is A = 4(125) - 2  
 A = 500 - 2  
 A = <<500-2=498>>498 pounds  
 Combined weight is 125 + 498 = <<125+498=623>>623 pounds  
 \boxed{623}<lim\_end>

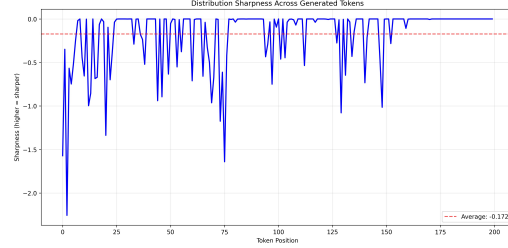
(a) SFT (example 1): Output with token-wise entropy overlay. Blue = low entropy (confident), red = high entropy (uncertain).



(b) SFT (example 1): Entropy per token position during decoding.

He gets 4\*12=\$<<4\*12=48>>48 a year  
 So he pays 15\*48=\$<<15\*48=720>>720  
 \boxed{720}<lim\_end>

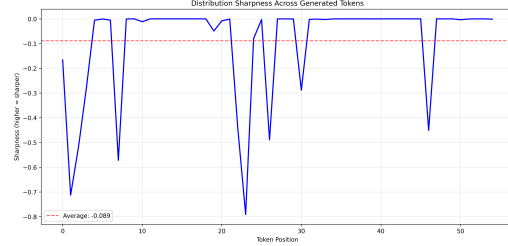
(c) RL (w/ KL, example 1): Output with token-wise entropy overlay.



(d) RL (w/ KL, example 1): Per-token entropy trend across output.

He gets 4\*12=\$<<4\*12=48>>48 a year  
 So he pays 15\*48=\$<<15\*48=720>>720  
 \boxed{720}<lim\_end>

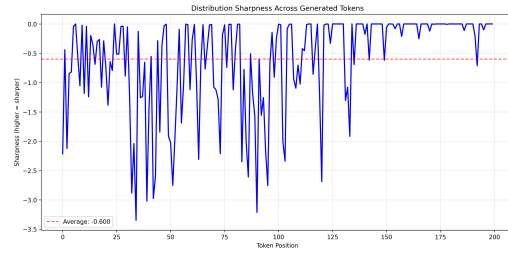
(e) SFT (example 2): Output with token-wise entropy overlay.



(f) SFT (example 2): Entropy plotted over decoding steps.

He gets 4\*12=\$<<4\*12=48>>48 a year  
 So he pays 15\*48=\$<<15\*48=720>>720  
 \boxed{720}<lim\_end>

(g) RL (w/ KL, example 2): Output with token-wise entropy overlay.



(h) RL (w/ KL, example 2): Token-level entropy curve.

Figure 5: Token-level entropy visualizations for Qwen3-1.7B. Each row compares SFT and RL (with KL) decoding behavior across two representative examples. Left: Entropy overlays on generated outputs, where red highlights uncertainty. Right: Corresponding token-wise entropy plotted over time. RL models exhibit sharper (lower entropy) predictions near answer tokens.

encourages exploration of different paths during training and rewards all paths that lead to correct answer, SFT on the other hand forces the model to collapse to one mode. (ii) As RL model gets closer to answer it gradually becomes more and more confident (the line gets closer to 0 and flatter). On the other hand for SFT model there is a dip in confidence just before they answer. For harder tasks, having higher entropy (larger negative values of  $e_t$ ) becomes important since it helps model explore and backtrack during inference to come to the correct solution. RL helping the model avoid a



mode/entropy collapse is likely one of the most important reasons why RL trained models are much better at harder tasks.

## 4 Discussion & Limitations

Our study focuses on two open-source models and a single math benchmark, which may limit generalization to other domains or model scales. While our structural analyses provide clear trends, they do not capture all possible forms of internal drift. Future work could expand to additional architectures, tasks, and interpretability tools for broader validation. A broader impact of the project is that most RL research usually focuses on better algorithms, evaluations and environments our project moves in a different direction and helps us understand why RL works.

## 5 Conclusion

This work provides a detailed comparison of how supervised fine-tuning (SFT) and reinforcement learning (RL) affect the internal representations of large language models. Using Qwen3-1.7B and LLaMA-3.2-1B on GSM8K, we find that RL significantly outperforms SFT in accuracy, while inducing smaller changes to model weights and preserving base structure.

SFT increases layer norms and rank, suggesting more memorization-driven behavior. In contrast, RL especially with KL regularization induces more targeted shifts, preserving pretrained attention patterns and directing focus toward task-critical tokens like `\boxed{}`. Token-level entropy and attention visualizations further support this interpretation, showing that RL leads to more stable and interpretable behavior.

These results suggest that RL fine-tuning encourages efficient and selective adaptation, while SFT drives more global but less structured changes. Understanding these internal differences is key to designing future alignment strategies that balance performance with interpretability.

## 6 Team Contributions

- **Rahul:** Led the interpretability and structural analysis experiments, including L2 distance, SVD rank tracking, attention visualizations, and entropy analysis. Also generated key plots and supported qualitative interpretation.
- **Arpan:** Focused on model training (SFT and RL via GRPO), evaluation across epochs, and experimental pipeline setup for both Qwen and LLaMA models.

**Changes from Proposal** While our initial plan focused on sparse autoencoders (SAEs) to analyze representational drift, we pivoted to a more scalable and rigorous methodology centered on structural and interpretability-driven analyses. This included layer-wise L2 norm shifts, SVD-based rank retention, token-level attention redistribution, and entropy dynamics across training regimes. We also transitioned to the GSM8K benchmark to support reward-based RL tuning and clearer interpretability. Though the approach evolved, our core objective of understanding how RL reshapes internal model structure remained central throughout.

## References

- Samira Abnar and Willem Zuidema. 2020. Quantifying Attention Flow in Transformers. *arXiv preprint arXiv:2005.00928* (2020).
- Federico Barbero, Álvaro Arroyo, Xiangming Gu, Christos Perivolaropoulos, Michael Bronstein, Petar Veličković, and Razvan Pascanu. 2025. Why do LLMs attend to the first token? *arXiv:2504.02732 [cs.CL]* <https://arxiv.org/abs/2504.02732>
- Yonatan Belinkov and James Glass. 2019. Analyzing and Interpreting Neural Networks for NLP. *NAACL* (2019).
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and

- John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168* (2021).
- Alex Cunningham and et al. 2023. Sparse Autoencoders for Disentangled Mechanisms in Transformers. *arXiv preprint arXiv:2310.00000* (2023).
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jia Shi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shutong Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanbiao Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948 [cs.CL]* <https://arxiv.org/abs/2501.12948>
- DeepSeek-AI. 2025. DeepSeek-R1. Hugging Face Model Card. Details open-source release of DeepSeek-R1 and R1-Zero models.
- DeepSeek-AI, Daya Guo, Dejian Yang, et al. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948* (2025). First RL-only fine-tuning, followed by cold-start SFT, achieving o1-level performance.
- Anonymous et al. 2024. DeepSeek-R1: Learning Complex Reasoning via RL-Based Fine-Tuning without Warm-Start. *arXiv preprint arXiv:2405.00000* (2024).
- John Hewitt and Christopher D. Manning. 2019. Structural Probing for Syntax in Word Representations. *Transactions of the ACL* (2019).
- X. Li, Y. Zhou, and Z. Chen. 2024. GeneralEntropy: Entropy-Regularized Fine-Tuning for Diverse LLM Outputs. *arXiv preprint arXiv:2404.01234* (2024).
- Daniel Meng, Leo Cosmo, Lu Lu, and et al. 2022. Locating and Editing Factual Associations in GPT. *arXiv preprint arXiv:2205.10059* (2022).
- Chris Olah, Elie Carter, et al. 2020. Zoom In: Mechanistic Interpretability in Neural Networks. *Distill* (2020).
- OpenAI. 2024a. Learning to Reason with LLMs: Introducing OpenAI o1. *OpenAI blog* (2024). OpenAI’s o1 can generate extended internal chains of thought before final output.
- OpenAI. 2024b. *OpenAI o1 System Card*. Technical Report. OpenAI. Describes the o1 reasoning model trained via reinforcement learning and chain-of-thought generation.



- Long Ouyang, Jeff Wu, Xu Jiang, and et al. 2022. Training Language Models to Follow Instructions with Human Feedback. *arXiv preprint arXiv:2203.02155* (2022).
- Zhihong Shao, Peiyi Wang, Qihao Zhu, and et al. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300* (2024).
- X. Wang and et al. 2023. Interpretability and Causal Tracing in Language Models. *arXiv preprint arXiv:2302.08194* (2023).
- Jason Wei, Maarten Bosma, Vincent Zhao, and et al. 2021. Finetuned Language Models for Zero-Shot Generalization. *arXiv preprint arXiv:2109.01652* (2021).
- Emily Wu and et al. 2023. Rewards Shape Model Attention: Evidence from RLHF. *arXiv preprint arXiv:2312.00000* (2023).